



```
In [1]: import os
import gc
import pandas as pd
import numpy as np
import re
from bs4 import BeautifulSoup
from nltk.tokenize import word_tokenize
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.linear_model import Perceptron
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import torch
from torch import nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
nltk.download("wordnet")
nltk.download("omw-1.4")
nltk.download("stopwords")
nltk.download('punkt')

seed = 42
torch.manual_seed(seed)
np.random.seed(seed)
```

```
[nltk_data] Downloading package wordnet to /Users/paul/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /Users/paul/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package stopwords to /Users/paul/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /Users/paul/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

1. Dataset Generation

```
In [2]: # read in data to dataframe
df = pd.read_csv('data.tsv', sep='\t', engine="python", on_bad_lines="skip")
print(df.columns)
```

```
Index(['marketplace', 'customer_id', 'review_id', 'product_id',
      'product_parent', 'product_title', 'product_category', 'star_rating',
      'helpful_votes', 'total_votes', 'vine', 'verified_purchase',
      'review_headline', 'review_body', 'review_date'],
      dtype='str')
```

```
In [3]: # sample 50k rows of each rating
n = 50000
df = (
    df.groupby("star_rating", group_keys=False)
```

```

        .sample(n=n, random_state=42)
    )
    print(df["star_rating"].value_counts())

```

star_rating

1	50000
2	50000
3	50000
4	50000
5	50000

Name: count, dtype: int64

```

In [4]: # create new integer column sentiment where:
# ratings > 3 is class 1
# ratings < 3 is class 2
# ratings = 3 is class 3
df["sentiment"] = -1
df.loc[df["star_rating"] > 3, "sentiment"] = 1
df.loc[df["star_rating"] < 3, "sentiment"] = 2
df.loc[df["star_rating"] == 3, "sentiment"] = 3
print(df["sentiment"].value_counts())

```

sentiment

2	100000
1	100000
3	50000

Name: count, dtype: int64

2. Word Embedding

```

In [5]: """
2a) Load model
"""

import gensim.downloader as api
from gensim.models import KeyedVectors

model: KeyedVectors = api.load('word2vec-google-news-300') # type: ignore

```

```

In [6]: # check semantic similarities with word2vec-google-news-300 model
def v(word):
    return model.get_vector(word)

vec = v("puppy") - v("dog") + v("cat")
print([model.similar_by_vector(vec, topn=5)])

vec = v("horrid")
print(model.similar_by_vector(vec, topn=5))

```

[('cat', 0.8460758924484253), ('puppy', 0.8246198296546936), ('kitten', 0.7781847715377808), ('puppies', 0.7319909334182739), ('pup', 0.7162657976150513)]

[('horrid', 0.9999998807907104), ('dreadful', 0.8229274153709412), ('atrocious', 0.7765751481056213), ('horrible', 0.7669703960418701), ('horrendous', 0.7628019452095032)]

```

In [7]: """
2b) Train Word2Vec model using my dataset
"""

from gensim.models import Word2Vec

"""Data Cleaning"""
# convert all reviews to lowercase
df["review_body"] = df["review_body"].str.lower().fillna('').astype(str)

def strip_html_and_urls(text):
    if pd.isna(text):
        return text
    # Remove HTML
    text = BeautifulSoup(text, "html.parser").get_text()
    # Remove URLs
    text = re.sub(r'http\S+|www.\S+', '', text)
    return text

# remove extra spaces
df["review_body"] = df["review_body"].str.replace(r"\s+", " ", regex=True)

# perform contractions
contractions = {
    "won't": "will not",
    "wouldn't": "would not",
    "can't": "cannot",
    "couldn't": "could not",
    "i'm": "i am",
    "ain't": "am not",
    "it's": "it is",
    "you're": "you are",
    "they're": "they are",
    "i've": "i have",
    "isn't": "is not",
    "aren't": "are not",
    "don't": "do not",
    "didn't": "did not",
    "doesn't": "does not",
    "musn't": "must not",
    "hasn't": "has not",
    "hadn't": "had not",
    "haven't": "have not"
}

# add the lazy version of contractions (without apostrophe)
# to dict
for k in list(contractions.keys()):
    contractions[k.replace("'", "")] = contractions[k]

def expand_contractions(text):
    if pd.isna(text):
        return text
    pattern = re.compile(r'\b(' + '|'.join(contractions.keys()) + r')\b')

```

```

    return pattern.sub(lambda x: contractions[x.group()], text)

df["review_body"] = df["review_body"].apply(expand_contractions)

# remove non-alphabetical characters
df["review_body"] = df["review_body"].str.replace(r"[^a-zA-Z\s]", "", regex=True)
tokenized_reviews = df['review_body'].apply(word_tokenize).tolist()

"""Train model (or load if already trained, as training takes ~4mins+)"""
model_path = './word2vec_custom.model'

if os.path.exists(model_path):
    # Load the existing model
    print("Loading model...")
    my_model = Word2Vec.load(model_path)
else:
    # Train a new model
    print("Training model...")
    my_model = Word2Vec(
        sentences=tokenized_reviews,
        vector_size=300, # embedding size
        window=11,
        min_count=10,
        sg=1
    )
    my_model.save(model_path)

```

Loading model...

In [8]: `# check semantic similarities with my model`

```

def v1(word):
    return my_model.wv[word]

vec = v1("puppy") - v1("dog") + v1("cat")
print([my_model.wv.most_similar(vec, topn=5)])

vec = v1("horrid")
print(my_model.wv.most_similar(vec, topn=5))

```

```

[('cat', 0.6703614592552185), ('puppy', 0.5569714307785034), ('litter', 0.3765
2987241744995), ('yoga', 0.3554334342479706), ('cats', 0.35217171907424927)]
[('horrid', 1.0), ('dreadful', 0.5951886177062988), ('horrendous', 0.5678995251
655579), ('terrible', 0.5592218637466431), ('aweful', 0.5539254546165466)]

```

3. Simple Models

In [9]: `"""Data preprocessing"""`

```

# stop word removal
stop_words = set(stopwords.words("english"))
words_to_keep = {
    "not", "no",
    "will", "would", "can", "could", "am", "is", "are",

```

```

    "have", "has", "had", "must", "do", "does", "did"
}
stop_words -= words_to_keep
df["review_body"] = df["review_body"].apply(
    lambda x: " ".join([word for word in x.split() if word not in stop_words])
    if isinstance(x, str) else ""
)

# lemmatization
lemmatizer = WordNetLemmatizer()
df["review_body"] = df["review_body"].apply(lambda x: " ".join([lemmatizer.lem

"""Compute average vector"""
def average(review_body, wv):
    token_list = review_body.split()
    vectors = [wv[word] for word in token_list if word in wv]

    if len(vectors) == 0:
        return np.zeros(wv.vector_size)

    return np.mean(vectors, axis=0)

data_df = df[df["sentiment"].isin([1, 2])]
X = np.vstack(data_df["review_body"].apply(lambda x: average(x, my_model.wv)).
X_pretrained = np.vstack(data_df["review_body"].apply(lambda x: average(x, mod
y_bin = np.array(data_df["sentiment"]))

# split binary dataset into training and testing
X_train, X_test, X_train_pre, X_test_pre, y_train, y_test = train_test_split(
    X, X_pretrained, y_bin, test_size=0.2, random_state=42, stratify=y_bin
)

```

```

In [10]: """Train perceptron model on my features"""
perceptron = Perceptron(
    max_iter=1000,
    tol=1e-3,
    random_state=42
)
perceptron.fit(X_train, y_train)

"""Train SVM model on my features"""
svm = LinearSVC(
    random_state=42,
    max_iter=10000
)

svm.fit(X_train, y_train)

"""Train perceptron model on pretrained features"""
perceptron_pre = Perceptron(
    max_iter=1000,
    tol=1e-3,
    random_state=42
)

```

```

)
perceptron_pre.fit(X_train_pre, y_train)

"""Train SVM model on pretrained features"""
svm_pre = LinearSVC(
    random_state=42,
    max_iter=10000
)

svm_pre.fit(X_train_pre, y_train)

```

Out[10]:

▼ LinearSVC ⓘ ?	
Parameters	
penalty	'l2'
loss	'squared_hinge'
dual	'auto'
tol	0.0001
C	1.0
multi_class	'ovr'
fit_intercept	True
intercept_scaling	1
class_weight	None
verbose	0
random_state	42
max_iter	10000

In [11]:

```

"""Report accuracy"""
y_test_perceptron = perceptron.predict(X_test)
y_test_perceptron_pre = perceptron_pre.predict(X_test_pre)
y_test_svm = svm.predict(X_test)
y_test_svm_pre = svm_pre.predict(X_test_pre)

test_acc_perceptron = accuracy_score(y_test, y_test_perceptron)
test_acc_perceptron_pre = accuracy_score(y_test, y_test_perceptron_pre)
test_acc_svm = accuracy_score(y_test, y_test_svm)
test_acc_svm_pre = accuracy_score(y_test, y_test_svm_pre)

print("Perceptron test accuracy:", test_acc_perceptron)
print("Perceptron pretrained test accuracy:", test_acc_perceptron_pre)
print("SVM test accuracy:", test_acc_svm)
print("SVM pretrained test accuracy:", test_acc_svm_pre)

```

Perceptron test accuracy: 0.81875
Perceptron pretrained test accuracy: 0.783475
SVM test accuracy: 0.849425
SVM pretrained test accuracy: 0.834325

4. Feedforward Neural Networks

```
In [12]: class MLP(nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 50),
            nn.ReLU(),
            nn.Linear(50, 10),
            nn.ReLU(),
            nn.Linear(10, output_dim)
        )

    def forward(self, x):
        return self.model(x)

    """4a) Averaged vectors"""
    # create binary datasets
    df_bin = df[df["sentiment"].isin([1, 2])]
    sentiment_bin = df_bin["sentiment"] - 1 # remap class labels to 0 and 1
    X_bin = np.vstack([average(x, my_model.wv) for x in df_bin["review_body"]]) #
    X_pre_bin = np.vstack([average(x, model) for x in df_bin["review_body"]]) # ty
    y_bin = np.array(sentiment_bin)

    # split binary dataset into train test split
    X_train_bin, X_test_bin, X_train_pre_bin, X_test_pre_bin, y_train_bin, y_test_
        X_bin, X_pre_bin, y_bin, test_size=0.2, random_state=42, stratify=y_bin
    )

    # convert binary to tensors
    X_train_bin = torch.from_numpy(X_train_bin).float()
    X_train_pre_bin = torch.from_numpy(X_train_pre_bin).float()
    y_train_bin = torch.from_numpy(y_train_bin).long()

    X_test_bin = torch.from_numpy(X_test_bin).float()
    X_test_pre_bin = torch.from_numpy(X_test_pre_bin).float()
    y_test_bin = torch.from_numpy(y_test_bin).long()

    # create binary tensor datasets
    train_dataset_bin = TensorDataset(X_train_bin, y_train_bin)
    test_dataset_bin = TensorDataset(X_test_bin, y_test_bin)
    train_dataset_pre_bin = TensorDataset(X_train_pre_bin, y_train_bin)
    test_dataset_pre_bin = TensorDataset(X_test_pre_bin, y_test_bin)

    # create binary dataloaders
    train_loader_bin = DataLoader(train_dataset_bin, batch_size=32, shuffle=True)
    test_loader_bin = DataLoader(test_dataset_bin, batch_size=32)
```

```

train_loader_pre_bin = DataLoader(train_dataset_pre_bin, batch_size=32, shuffle=True)
test_loader_pre_bin = DataLoader(test_dataset_pre_bin, batch_size=32)

# create ternary datasets
X_tern = np.vstack([average(x, my_model.wv) for x in df["review_body"]]) # type: np.ndarray
X_pre_tern = np.vstack([average(x, model) for x in df["review_body"]]) # type: np.ndarray
sentiment_tern = df["sentiment"] - 1 # remap class labels to 0,1,2
y_tern = np.array(sentiment_tern)

# split ternary dataset into train test split
X_train_tern, X_test_tern, X_train_pre_tern, X_test_pre_tern, y_train_tern, y_test_tern = \
    train_test_split(
        X_tern,
        X_pre_tern,
        y_tern,
        test_size=0.2,
        random_state=42,
        stratify=y_tern
    )

# convert ternary to tensors
X_train_tern = torch.from_numpy(X_train_tern).float()
X_train_pre_tern = torch.from_numpy(X_train_pre_tern).float()
y_train_tern = torch.from_numpy(y_train_tern).long()

X_test_tern = torch.from_numpy(X_test_tern).float()
X_test_pre_tern = torch.from_numpy(X_test_pre_tern).float()
y_test_tern = torch.from_numpy(y_test_tern).long()

# create ternary tensor datasets
train_dataset_tern = TensorDataset(X_train_tern, y_train_tern)
test_dataset_tern = TensorDataset(X_test_tern, y_test_tern)

train_dataset_pre_tern = TensorDataset(X_train_pre_tern, y_train_tern)
test_dataset_pre_tern = TensorDataset(X_test_pre_tern, y_test_tern)

# create ternary dataloaders
train_loader_tern = DataLoader(train_dataset_tern, batch_size=32, shuffle=True)
test_loader_tern = DataLoader(test_dataset_tern, batch_size=32)

train_loader_pre_tern = DataLoader(train_dataset_pre_tern, batch_size=32, shuffle=True)
test_loader_pre_tern = DataLoader(test_dataset_pre_tern, batch_size=32)

```

In [13]: `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`

```

"""Train binary nn model on custom averaged features"""
mlp_bin = MLP(input_dim=X_bin.shape[1], output_dim=2).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_bin.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')

```



```

mlp_bin.train()
for xb, yb in train_loader_bin:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad()
    outputs = mlp_bin(xb)
    loss = criterion(outputs, yb)
    loss.backward()
    optimizer.step()

```

Training epoch 20...

```

In [14]: """Train binary nn model on pretrained averaged features"""
mlp_pre_bin = MLP(input_dim=X_pre_bin.shape[1], output_dim=2).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_pre_bin.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_pre_bin.train()
    for xb, yb in train_loader_pre_bin:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_pre_bin(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

```

Training epoch 20...

```

In [15]: """Train ternary nn model on custom averaged features"""
mlp_tern = MLP(input_dim=X_tern.shape[1], output_dim=3).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_tern.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_tern.train()
    for xb, yb in train_loader_tern:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_tern(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

```

Training epoch 20...

```

In [16]: """Train ternary nn model on pretrained averaged features"""
mlp_pre_tern = MLP(input_dim=X_pre_tern.shape[1], output_dim=3).to(device)

criterion = nn.CrossEntropyLoss()

```

```
optimizer = torch.optim.Adam(mlp_pre_tern.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_pre_tern.train()
    for xb, yb in train_loader_pre_tern:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_pre_tern(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()
```

Training epoch 20...

```
In [17]: """Report accuracies for NN (averaged)"""
mlp_bin.eval()
with torch.no_grad():
    mlp_bin_logits = mlp_bin(X_test_bin.to(device))
    mlp_bin_preds = torch.argmax(mlp_bin_logits, dim=1)
    mlp_bin_accuracy = (mlp_bin_preds == y_test_bin).float().mean().item()

mlp_pre_bin.eval()
with torch.no_grad():
    mlp_pre_bin_logits = mlp_pre_bin(X_test_pre_bin.to(device))
    mlp_pre_bin_preds = torch.argmax(mlp_pre_bin_logits, dim=1)
    mlp_pre_bin_accuracy = (mlp_pre_bin_preds == y_test_bin).float().mean().it

mlp_tern.eval()
with torch.no_grad():
    mlp_tern_logits = mlp_tern(X_test_tern.to(device))
    mlp_tern_preds = torch.argmax(mlp_tern_logits, dim=1)
    mlp_tern_accuracy = (mlp_tern_preds == y_test_tern).float().mean().item()

mlp_pre_tern.eval()
with torch.no_grad():
    mlp_pre_tern_logits = mlp_pre_tern(X_test_pre_tern.to(device))
    mlp_pre_tern_preds = torch.argmax(mlp_pre_tern_logits, dim=1)
    mlp_pre_tern_accuracy = (mlp_pre_tern_preds == y_test_tern).float().mean()

print("NN custom binary test accuracy:", mlp_bin_accuracy)
print("NN pretrained binary test accuracy:", mlp_pre_bin_accuracy)
print("NN custom ternary test accuracy:", mlp_tern_accuracy)
print("NN pretrained ternary test accuracy:", mlp_pre_tern_accuracy)
```

NN custom binary test accuracy: 0.871025025844574
 NN pretrained binary test accuracy: 0.8593249917030334
 NN custom ternary test accuracy: 0.707260012626648
 NN pretrained ternary test accuracy: 0.6944800019264221

```
In [18]: """4b) Concatenated vectors"""
def concat(review_body, wv, max_words):
    token_list = review_body.split()
```

```

vectors = [wv[word] for word in token_list if word in wv]

# Pad with zeros if length < max_words
if len(vectors) < max_words:
    padding = [np.zeros(wv.vector_size) for _ in range(max_words - len(vectors))]
    vectors.extend(padding)
else:
    vectors = vectors[:max_words]

return np.concatenate(vectors, axis=0)

# create binary datasets
X_bin = np.stack([concat(x, my_model.wv, 10) for x in df_bin["review_body"]])
X_pre_bin = np.stack([concat(x, model, 10) for x in df_bin["review_body"]]) # type: ignore
y_bin = np.array(sentiment_bin)

# split binary dataset into train test split
X_train_bin, X_test_bin, X_train_pre_bin, X_test_pre_bin, y_train_bin, y_test_bin = \
    train_test_split(X_bin, X_pre_bin, y_bin, test_size=0.2, random_state=42, stratify=y_bin)

# convert binary to tensors
X_train_bin = torch.from_numpy(X_train_bin).float()
X_train_pre_bin = torch.from_numpy(X_train_pre_bin).float()
y_train_bin = torch.from_numpy(y_train_bin).long()

X_test_bin = torch.from_numpy(X_test_bin).float()
X_test_pre_bin = torch.from_numpy(X_test_pre_bin).float()
y_test_bin = torch.from_numpy(y_test_bin).long()

# create binary tensor datasets
train_dataset_bin = TensorDataset(X_train_bin, y_train_bin)
test_dataset_bin = TensorDataset(X_test_bin, y_test_bin)

train_dataset_pre_bin = TensorDataset(X_train_pre_bin, y_train_bin)
test_dataset_pre_bin = TensorDataset(X_test_pre_bin, y_test_bin)

# create binary dataloaders
train_loader_bin = DataLoader(train_dataset_bin, batch_size=32, shuffle=True)
test_loader_bin = DataLoader(test_dataset_bin, batch_size=32)
train_loader_pre_bin = DataLoader(train_dataset_pre_bin, batch_size=32, shuffle=True)
test_loader_pre_bin = DataLoader(test_dataset_pre_bin, batch_size=32)

# create ternary datasets
X_tern = np.stack([concat(x, my_model.wv, 10) for x in df["review_body"]]) # type: ignore
X_pre_tern = np.stack([concat(x, model, 10) for x in df["review_body"]]) # type: ignore
sentiment_tern = df["sentiment"] - 1 # remap class labels to 0,1,2
y_tern = np.array(sentiment_tern)

# split ternary dataset into train test split
X_train_tern, X_test_tern, X_train_pre_tern, X_test_pre_tern, y_train_tern, y_test_tern = \
    train_test_split(X_tern, X_pre_tern, y_tern, test_size=0.2, random_state=42, stratify=y_tern)

```

```

    y_tern,
    test_size=0.2,
    random_state=42,
    stratify=y_tern
)

# convert ternary to tensors
X_train_tern = torch.from_numpy(X_train_tern).float()
X_train_pre_tern = torch.from_numpy(X_train_pre_tern).float()
y_train_tern = torch.from_numpy(y_train_tern).long()

X_test_tern = torch.from_numpy(X_test_tern).float()
X_test_pre_tern = torch.from_numpy(X_test_pre_tern).float()
y_test_tern = torch.from_numpy(y_test_tern).long()

# create ternary tensor datasets
train_dataset_tern = TensorDataset(X_train_tern, y_train_tern)
test_dataset_tern = TensorDataset(X_test_tern, y_test_tern)

train_dataset_pre_tern = TensorDataset(X_train_pre_tern, y_train_tern)
test_dataset_pre_tern = TensorDataset(X_test_pre_tern, y_test_tern)

# create ternary dataloaders
train_loader_tern = DataLoader(train_dataset_tern, batch_size=32, shuffle=True)
test_loader_tern = DataLoader(test_dataset_tern, batch_size=32)

train_loader_pre_tern = DataLoader(train_dataset_pre_tern, batch_size=32, shuffle=True)
test_loader_pre_tern = DataLoader(test_dataset_pre_tern, batch_size=32)

```

```

In [19]: """Train binary nn model on custom concatenated features"""
mlp_bin_concat = MLP(input_dim=X_bin.shape[1], output_dim=2).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_bin_concat.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_bin_concat.train()
    for xb, yb in train_loader_bin:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_bin_concat(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

```

Training epoch 20...

```

In [20]: """Train binary nn model on pretrained concatenated features"""
mlp_pre_bin_concat = MLP(input_dim=X_bin.shape[1], output_dim=2).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_pre_bin_concat.parameters(), lr=0.001)

```

```

num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_pre_bin_concat.train()
    for xb, yb in train_loader_pre_bin:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_pre_bin_concat(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

```

Training epoch 20...

```

In [21]: """Train ternary nn model on custom concatenated features"""
mlp_tern_concat = MLP(input_dim=X_tern.shape[1], output_dim=3).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_tern_concat.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_tern_concat.train()
    for xb, yb in train_loader_tern:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_tern_concat(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

```

Training epoch 20...

```

In [22]: """Train ternary nn model on pretrained concatenated features"""
mlp_pre_tern_concat = MLP(input_dim=X_tern.shape[1], output_dim=3).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp_pre_tern_concat.parameters(), lr=0.001)
num_epochs = 20

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    mlp_pre_tern_concat.train()
    for xb, yb in train_loader_pre_tern:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = mlp_pre_tern_concat(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

```

Training epoch 20...

```

In [23]: """Report accuracies for NN (concatenated)"""
mlp_bin_concat.eval()
with torch.no_grad():
    mlp_bin_concat_logits = mlp_bin_concat(X_test_bin.to(device))
    mlp_bin_concat_preds = torch.argmax(mlp_bin_concat_logits, dim=1)
    mlp_bin_concat_accuracy = (mlp_bin_concat_preds == y_test_bin).float().mean()

mlp_pre_bin_concat.eval()
with torch.no_grad():
    mlp_pre_bin_concat_logits = mlp_pre_bin_concat(X_test_pre_bin.to(device))
    mlp_pre_bin_concat_preds = torch.argmax(mlp_pre_bin_concat_logits, dim=1)
    mlp_pre_bin_concat_accuracy = (mlp_pre_bin_concat_preds == y_test_bin).float().mean()

mlp_tern_concat.eval()
with torch.no_grad():
    mlp_tern_concat_logits = mlp_tern_concat(X_test_tern.to(device))
    mlp_tern_concat_preds = torch.argmax(mlp_tern_concat_logits, dim=1)
    mlp_tern_concat_accuracy = (mlp_tern_concat_preds == y_test_tern).float().mean()

mlp_pre_tern_concat.eval()
with torch.no_grad():
    mlp_pre_tern_concat_logits = mlp_pre_tern_concat(X_test_pre_tern.to(device))
    mlp_pre_tern_concat_preds = torch.argmax(mlp_pre_tern_concat_logits, dim=1)
    mlp_pre_tern_concat_accuracy = (mlp_pre_tern_concat_preds == y_test_tern).float().mean()

# free memory
del X_bin, X_pre_bin, y_bin
del X_tern, X_pre_tern, y_tern

del X_train_bin, X_test_bin, X_train_pre_bin, X_test_pre_bin
del X_train_tern, X_test_tern, X_train_pre_tern, X_test_pre_tern

del y_train_bin, y_test_bin
del y_train_tern, y_test_tern

del train_dataset_bin, test_dataset_bin
del train_dataset_pre_bin, test_dataset_pre_bin
del train_dataset_tern, test_dataset_tern
del train_dataset_pre_tern, test_dataset_pre_tern

del train_loader_bin, test_loader_bin
del train_loader_pre_bin, test_loader_pre_bin
del train_loader_tern, test_loader_tern
del train_loader_pre_tern, test_loader_pre_tern

# clear GPU cache if using CUDA
if torch.cuda.is_available():
    torch.cuda.empty_cache()

print("NN custom binary (concatenated) test accuracy:", mlp_bin_concat_accuracy)
print("NN pretrained binary (concatenated) test accuracy:", mlp_pre_bin_concat_accuracy)
print("NN custom ternary (concatenated) test accuracy:", mlp_tern_concat_accuracy)
print("NN pretrained ternary (concatenated) test accuracy:", mlp_pre_tern_concat_accuracy)

```

NN custom binary (concatenated) test accuracy: 0.7637249827384949
NN pretrained binary (concatenated) test accuracy: 0.7600499987602234
NN custom ternary (concatenated) test accuracy: 0.5982199907302856
NN pretrained ternary (concatenated) test accuracy: 0.5855000019073486

5. Convolutional Neural Networks

```
In [ ]: class CNN(nn.Module):
    def __init__(self, embedding_dim, num_classes):
        super().__init__()
        self.conv1 = nn.Conv1d(in_channels=embedding_dim, out_channels=50, kernel_size=3)
        self.conv2 = nn.Conv1d(in_channels=50, out_channels=10, kernel_size=3)
        self.fc = nn.Linear(10 * 50, num_classes)

    def forward(self, x):
        x = x.transpose(1, 2)
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = x.flatten(start_dim=1)
        x = self.fc(x)
        return x

    def cnn_input(review, ww, max_words):
        tokens = review.split()
        vectors = [ww[word] for word in tokens if word in ww][:max_words]
        arr = np.zeros((max_words, ww.vector_size), dtype=np.float32)
        if vectors:
            arr[:len(vectors)] = vectors
        return arr
```

```
In [ ]: """Train binary cnn model on custom features"""
X_bin = np.stack([cnn_input(x, my_model.ww, 50) for x in df_bin["review_body"]])
y_bin = np.array(sentiment_bin)

X_train_bin, X_test_bin, y_train_bin, y_test_bin = train_test_split(
    X_bin, y_bin, test_size=0.2, random_state=42, stratify=y_bin
)

X_train_bin = torch.from_numpy(X_train_bin).float()
X_test_bin = torch.from_numpy(X_test_bin).float()
y_train_bin = torch.from_numpy(y_train_bin).long()
y_test_bin = torch.from_numpy(y_test_bin).long()

train_dataset_bin = TensorDataset(X_train_bin, y_train_bin)
test_dataset_bin = TensorDataset(X_test_bin, y_test_bin)

train_loader_bin = DataLoader(train_dataset_bin, batch_size=32, shuffle=True)
test_loader_bin = DataLoader(test_dataset_bin, batch_size=32)

cnn_bin = CNN(embedding_dim=my_model.ww.vector_size, num_classes=2).to(device)
```

```

model_path = './cnn_bin.pth'

if os.path.exists(model_path):
    # Load the existing model
    print("Loading model...")
    cnn_bin.load_state_dict(torch.load(model_path))
else:
    # Train a new model
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(cnn_bin.parameters(), lr=0.001)
    num_epochs = 10

    # train
    for epoch in range(num_epochs):
        print(f"Training epoch {epoch+1}...", end='\r')
        cnn_bin.train()
        for xb, yb in train_loader_bin:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()

            outputs = cnn_bin(xb)
            loss = criterion(outputs, yb)
            loss.backward()
            optimizer.step()

        # save model
        torch.save(cnn_bin.state_dict(), model_path)

    # calculate accuracy
    cnn_bin.eval()
    with torch.no_grad():
        cnn_bin_logits = cnn_bin(X_test_bin.to(device))
        cnn_bin_preds = torch.argmax(cnn_bin_logits, dim=1)
        cnn_bin_accuracy = (cnn_bin_preds == y_test_bin).float().mean().item()

    # free memory
    del X_bin
    del X_train_bin, X_test_bin, y_train_bin, y_test_bin
    del train_dataset_bin, test_dataset_bin, train_loader_bin, test_loader_bin
    torch.cuda.empty_cache()
    gc.collect()

```

Loading model...

Out[]: 31

```

In [29]: """Train binary cnn model on pretrained features"""
X_pre_bin = np.stack([cnn_input(x, model, 50) for x in df_bin["review_body"]])

X_train_pre_bin, X_test_pre_bin, y_train_bin, y_test_bin = train_test_split(
    X_pre_bin, y_bin, test_size=0.2, random_state=42, stratify=y_bin
)

X_train_pre_bin = torch.from_numpy(X_train_pre_bin)

```



```

X_test_pre_bin = torch.from_numpy(X_test_pre_bin)

train_dataset_pre_bin = TensorDataset(X_train_pre_bin, y_train_bin)
test_dataset_pre_bin = TensorDataset(X_test_pre_bin, y_test_bin)

train_loader_pre_bin = DataLoader(train_dataset_pre_bin, batch_size=32, shuffle=True)
test_loader_pre_bin = DataLoader(test_dataset_pre_bin, batch_size=32)

cnn_pre_bin = CNN(embedding_dim=model.vector_size, num_classes=2).to(device)

model_path = './cnn_pre_bin.pth'

if os.path.exists(model_path):
    # Load the existing model
    print("Loading model...")
    cnn_pre_bin.load_state_dict(torch.load(model_path))
else:
    # Train a new model
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(cnn_pre_bin.parameters(), lr=0.001)
    num_epochs = 10

    # train
    for epoch in range(num_epochs):
        print(f"Training epoch {epoch+1}...", end='\r')
        cnn_pre_bin.train()
        for xb, yb in train_loader_pre_bin:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            outputs = cnn_pre_bin(xb)
            loss = criterion(outputs, yb)
            loss.backward()
            optimizer.step()

    # save model
    torch.save(cnn_pre_bin.state_dict(), model_path)

# calculate accuracy
cnn_pre_bin.eval()
with torch.no_grad():
    cnn_pre_bin_logits = cnn_pre_bin(X_test_pre_bin.to(device))
    cnn_pre_bin_preds = torch.argmax(cnn_pre_bin_logits, dim=1)
    cnn_pre_bin_accuracy = (cnn_pre_bin_preds == y_test_bin).float().mean()

# free memory
del X_pre_bin
del X_train_pre_bin, X_test_pre_bin
del train_dataset_pre_bin, test_dataset_pre_bin
del train_loader_pre_bin, test_loader_pre_bin
torch.cuda.empty_cache()
gc.collect()

```

Loading model...

Out[29]: 1454

```
In [30]: """Train ternary cnn model on custom features"""
X_tern = np.stack([cnn_input(x, my_model.wv, 50) for x in df["review_body"]])
y_tern = np.array(df["sentiment"] - 1) # remap to 0,1,2

# train-test split
X_train_tern, X_test_tern, y_train_tern, y_test_tern = train_test_split(
    X_tern, y_tern, test_size=0.2, random_state=42, stratify=y_tern
)

# convert to tensors
X_train_tern = torch.from_numpy(X_train_tern).float()
X_test_tern = torch.from_numpy(X_test_tern).float()
y_train_tern = torch.from_numpy(y_train_tern).long()
y_test_tern = torch.from_numpy(y_test_tern).long()

# create datasets and loaders
train_dataset_tern = TensorDataset(X_train_tern, y_train_tern)
test_dataset_tern = TensorDataset(X_test_tern, y_test_tern)
train_loader_tern = DataLoader(train_dataset_tern, batch_size=32, shuffle=True)
test_loader_tern = DataLoader(test_dataset_tern, batch_size=32)

cnn_tern = CNN(embedding_dim=my_model.wv.vector_size, num_classes=3).to(device)

model_path = './cnn_tern.pth'

if os.path.exists(model_path):
    # Load the existing model
    print("Loading model...")
    cnn_tern.load_state_dict(torch.load(model_path))
else:
    # Train a new model
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(cnn_tern.parameters(), lr=0.001)
    num_epochs = 10

    # train
    for epoch in range(num_epochs):
        print(f"Training epoch {epoch+1}...", end='\r')
        cnn_tern.train()
        for xb, yb in train_loader_tern:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            outputs = cnn_tern(xb)
            loss = criterion(outputs, yb)
            loss.backward()
            optimizer.step()

    # save model
    torch.save(cnn_tern.state_dict(), model_path)

# calculate accuracy
```

```

cnn_tern.eval()
with torch.no_grad():
    cnn_tern_logits = cnn_tern(X_test_tern.to(device))
    cnn_tern_preds = torch.argmax(cnn_tern_logits, dim=1)
    cnn_tern_accuracy = (cnn_tern_preds == y_test_tern).float().mean().item()

# free memory
del X_tern, y_tern
del X_train_tern, X_test_tern, y_train_tern, y_test_tern
del train_dataset_tern, test_dataset_tern, train_loader_tern, test_loader_tern
torch.cuda.empty_cache()
gc.collect()

```

Training epoch 10...

Out[30]: 9

```

In [31]: """Train ternary cnn model on pretrained features"""
X_pre_tern = np.stack([cnn_input(x, model, 50) for x in df["review_body"]])
y_tern = np.array(df["sentiment"] - 1) # remap to 0,1,2

# train-test split
X_train_pre_tern, X_test_pre_tern, y_train_tern, y_test_tern = train_test_split(
    X_pre_tern, y_tern, test_size=0.2, random_state=42, stratify=y_tern
)

# convert to tensors
X_train_pre_tern = torch.from_numpy(X_train_pre_tern).float()
X_test_pre_tern = torch.from_numpy(X_test_pre_tern).float()
y_train_tern = torch.from_numpy(y_train_tern).long()
y_test_tern = torch.from_numpy(y_test_tern).long()

# create datasets and loaders
train_dataset_pre_tern = TensorDataset(X_train_pre_tern, y_train_tern)
test_dataset_pre_tern = TensorDataset(X_test_pre_tern, y_test_tern)
train_loader_pre_tern = DataLoader(train_dataset_pre_tern, batch_size=32, shuffle=True)
test_loader_pre_tern = DataLoader(test_dataset_pre_tern, batch_size=32)

# initialize model
cnn_pre_tern = CNN(embedding_dim=model.vector_size, num_classes=3).to(device)

model_path = './cnn_pre_tern.pth'

if os.path.exists(model_path):
    # Load the existing model
    print("Loading model...")
    cnn_pre_tern.load_state_dict(torch.load(model_path))
else:
    # Train a new model
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(cnn_pre_tern.parameters(), lr=0.001)
    num_epochs = 10

    # train

```

```

for epoch in range(num_epochs):
    print(f"Training epoch {epoch+1}...", end='\r')
    cnn_pre_tern.train()
    for xb, yb in train_loader_pre_tern:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        outputs = cnn_pre_tern(xb)
        loss = criterion(outputs, yb)
        loss.backward()
        optimizer.step()

    # save model
    torch.save(cnn_pre_tern.state_dict(), model_path)

# calculate accuracy
cnn_pre_tern.eval()
with torch.no_grad():
    cnn_pre_tern_logits = cnn_pre_tern(X_test_pre_tern.to(device))
    cnn_pre_tern_preds = torch.argmax(cnn_pre_tern_logits, dim=1)
    cnn_pre_tern_accuracy = (cnn_pre_tern_preds == y_test_tern).float().mean()

# free memory
del X_pre_tern
del X_train_pre_tern, X_test_pre_tern, y_train_tern, y_test_tern
del train_dataset_pre_tern, test_dataset_pre_tern, train_loader_pre_tern, test
torch.cuda.empty_cache()
gc.collect()

```

Training epoch 10...

Out[31]: 9

```

In [32]: """Report accuracies for CNNs"""
print("CNN custom binary test accuracy:", cnn_bin_accuracy)
print("CNN pretrained binary test accuracy:", cnn_pre_bin_accuracy)
print("CNN custom ternary test accuracy:", cnn_tern_accuracy)
print("CNN pretrained ternary test accuracy:", cnn_pre_tern_accuracy)

```

CNN custom binary test accuracy: 0.880174994468689

CNN pretrained binary test accuracy: 0.7637249827384949

CNN custom ternary test accuracy: 0.7239599823951721

CNN pretrained ternary test accuracy: 0.7201399803161621